

Python Dictionary: key-value pairs

התרגול הזה עוסק בנושאים שראיתם בהרצאה 3 ובו נעבור על דוגמה לקוד ב-python שמראה איך עוברים מנתונים שמקבלים מתוכנת alignment, במקרה הזה מתוכנת שנקראת kallisto, לטבלה שאומרת לנו את רמות הביטוי של כל גן, כלומר כמה עותקים יש לנו מכל גן בדגימה.

המטרה של התרגול הזה היא לראות דוגמה לכתיבת קוד ב-python, להכיר משתנה שנקרא dictionary ואיך הוא מיושם ב-python, ולהתנסות עוד קצת ב-Regular Expression (RE), כמו שראינו בתרגול 1).

חזרה קצרה על דברים שלמדנו:

על מנת לקבל דאטא ולהתחיל לנתח אותו, יש לבצע את השלבים העיקריים הבאים:

1. חילוץ ה-RNA מהרקמה- הורסים פיזית את הרקמה ומחלצים ממנה את ה-RNA.
 - על מנת לבדוד את ה-RNA משאר הרקמה, ניתן להשתמש בשיטות שמבוססות על גודל, מטען (ה-RNA הוא בעל מטען שלילי), או מבנה ה-RNA (קשירה של polyA-polyT כמו שראיתם בהרצאה 2).
2. ריצוף של ה-RNA- קיימות מספר שיטות לריצוף ה-RNA, עד עכשיו דיברנו בעיקר על הטכנולוגיה של illumina (הרצאה 2).
 - לאחר שלב זה אנחנו מקבלים קובץ FASTQ שמכיל את הריצופים עם מידע על איכות הריצוף של כל בסיס (כמו שראינו בתרגול 1).
 - ניתן להיעזר ב-RE כדי להוריד אזורים בריצופים שלא קשורים לגנים (למשל UMI).
3. ביצוע alignment של הריצופים הנקיים לגנום או לטרנסקריפטום- קיימות מספר שיטות לביצוע alignment שמטרתן היא להתאים את הריצוף לאזור המתאים לו ביותר בגן וכך נוכל לדעת כמה עותקים מכל גן יש ברקמה, למשל bowtie2 שעושה alignment לגנום או לטרנסקריפטום ו-kallisto שעושה pseudo-alignment לטרנסקריפטום.

לאחר כל השלבים האלה, נוכל בסופו של דבר לקבל וקטור שמראה לנו כמה עותקים מכל גן היו בדגימה.

בתרגול הראשון דיברנו יותר בהרחבה על המעבר משלב 2 לשלב 3, כלומר איך נראה הדאטא אחרי מכונת הריצוף (קבצי FASTQ) ואיך עובדים עם RE. בתרגול הזה אנחנו נתמקד במעבר משלב 3 לקבלת וקטור רמות הביטוי של הגנים, כלומר כמה עותקים יש מכל גן ברקמה. אנחנו נלמד איך נראה הדאטא אחרי שלב ה-alignment שמתבצע כאן באמצעות kallisto ונראה באמצעות קוד python איך משלב 3 מגיעים לטבלה שמראה את רמות הביטוי של כל אחד מהגנים.

תזכורת על kallisto- תוכנה שעושה pseudo-alignment שלוקחת את כל הריצופים, משווה אותם לטרנסקריפטום ואומרת כמה ריצופים יש לכל גן. השיטה של kallisto לא תשווה ברמת הבסיס הבודד אלא היא תמצא את האזור והגן שהכי מתאים למקטע הריצוף. השיטות האלה חוסכות הרבה מאוד זמן וזיכרון, אך הן לא מאפשרות לגלות מודיפיקציות לעומת alignment מדויק, כמו bowtie2 למשל.

נעבור על קובץ התרגול:

הטבלה הראשונה היא מהאתר של kallisto והיא מראה לנו את הקשר בין שמות של טרנסקריפטים או איזופורמים לשמות של גנים. לכל גן יכולים כמה טרנסקריפטים/איזופורמים בגלל alternative splicing (וריאציות שונות של חיבורים של האקסונים של הגן). העמודה השמאלית בטבלה היא השם של הטרנסקריפט והעמודה הימנית היא השם של הגן (גם האמצעית אבל היא לא מעניינת אותנו הפעם). שימו לב שלכל גן יש מספר איזופורמים.

הטבלה השנייה היא התוצר של kallisto והיא מכילה עבור כל טרנסקריפט- את השם שלו, מה האורך שלו, מה ה-effective length (האורך של הטרנסקריפט פחות האורך של מקטע הריצוף. האורך של הטרנסקריפט משפיע על כמות הריצופים ששייכים אליו, ככל שהאורך שלו יהיה ארוך יותר, נקבל יותר מקטעי ריצוף ששייכים אליו), מה ה-estimated counts שלו (יש איזופורמים לגן ואם למשל יש לנו מקטע ריצוף קטן שיכול להתאים לכמה איזופורמים, אז לא נדע למי הוא שייך ולכן נשתמש באלגוריתם שישערך למי הוא שייך, למשל כמו EM (ראיתם בהרצאה 3 ונראה בתרגולים הבאים), ונקבל רק שיערוך של מספר העותקים שייכים לגן ולא מספר מדויק ולכן הוא יהיה שבר ולא מספר שלם), ומה ה-TPM שלו (transcripts per million, לוקח בחשבון את האורך של הטרנסקריפט ואת מספר הריצופים בדגימה כי הם משפיעים על כמות העותקים שאנחנו צופים לקבל לגן הזה. בקובץ התרגול יש את הנוסחה ל-TPM, ה-X זה הכמות של הגן וה-I זה האורך האפקטיבי של הגן).

מתי משתמשים ב-counts ומתי משתמשים ב-TPM?

מכיוון שהאורך של כל גן הוא שונה, אנחנו נצפה לקבל כמות שונה של מקטעי ריצוף שמשייכים אליו. למשל אנחנו נצפה לקבל יותר מקטעי ריצוף ששייכים לגן ארוך, למרות שלא בהכרח באמת יש יותר עותקים שלו בדגימה. לכן ה-counts עוזר לנו להשוות בין גנים ולמצוא גנים שרמות הביטוי שלהם לא קבועות (differentially expressed genes) אבל בתנאי שמתחשבים ברמות הביטוי של כל גן במספר דגימות שונות (למשל כמו בתרגיל בית 1, שממדלים את הדאטא לפי ממוצע ושונות של הגנים בדגימות השונות), ואילו ה-TPM מתחשב באורך של הגן ולכן הוא מאפשר לנו להשוות בין גנים שונים בתוך אותה דגימה.

כאשר נסתכל על דגימה אחת (כמו בדוגמה כאן) ונרצה לדעת כמה עותקים יש מגן מסוים, נצטרך לחבר את ערכי ה-TPM של כל הטרנסקריפטים שלו ולא לחבר את ה-counts, כי מספר העותקים מושפע מהאורך של הטרנסקריפט.

מה המטרה שלנו?

למצוא את רמות הביטוי של כל גן בדגימה. לשם כך, נחבר את ערכי ה-TPM של כל הטרנסקריפטים של כל אחד מהגנים. למשל 5 השורות הראשונות בקבצים אלו טרנסקריפטים ששייכים לאותו הגן, אז אנחנו נרצה לכתוב קוד שיחבר את כל ערכי ה-TPM שלהם.

איך עושים את זה?

נרצה לקרוא את הקובץ הראשון כדי שנדע לקשר בין שמות הטרנסקריפטים (עמודה שמאל) לשמות של הגנים (עמודה ימין). אח"כ נקרא את הקובץ השני, ובאמצעות הקשרים בקובץ הראשון, נקשר בין השמות של הטרנסקריפטים בקובץ השני לשמות של הגנים שלהם, ולבסוף נסכום את הכמות שלהם, את ה-TPM של כל הטרנסקריפטים ששייכים לאותו הגן, כך שנקבל את וקטור רמות הביטוי של הגנים ברקמה (כמו שרואים בעמוד האחרון של קובץ התרגול ב-Output file).

במה נעזר?

נשתמש גם ב-RE כדי לקרוא את 2 הקבצים האלה ולמצוא בהם דפוסים שמעניינים אותנו (כמו השם של הטרנסקריפט, השם של הגן וערך ה-TPM שלו) וגם ב-dictionary שיקשר לנו בין פרמטרים שונים (כמו השם של הטרנסקריפט לשם של הגן והשם של הגן לערך ה-TPM הכולל שלו).

מה זה dictionary?

בגדול כמו מילון רגיל שאתם מכירים. המילון מורכב מ-key ומ-value, כך שלכל key אנחנו מגדירים את ה-value שלו. למשל אנחנו יכולים להגדיר את ה-key ל-"sunflower", ואת ה-value שלו ל-"flower". הערכים ב-dictionary יכולים להיות כל סוג של משתנה (מספר, אותיות וכו').

ב-python אנחנו מגדירים את המשתנה dictionary עם סוגריים מסולסלים, למשל:

```
Dict = { }
```

כדי להכניס הגדרה חדשה למילון Dict, נשתמש בסוגריים מרובעים, למשל:

```
Dict ["sunflower"] = "flower"
```

עכשיו ברגע שנחפש את המילה sunflower במילון, נקבל את המילה flower, למשל:

```
Dict ["sunflower"]
```

```
flower
```

אם אנחנו רוצים לקשר בין השמות של הטרנסקריפטים לכמות שיש מכל גן, איזה מילונים נרצה ליצור?

אנחנו נרצה ליצור 2 מילונים.

- למילון הראשון נקרא gene_name_of_transcript ובו ה-key יהיה השם של הטרנסקריפט וה-value יהיה השם של הגן. באופן הזה נוכל לקשר בין השמות ולהשתמש בזה עבור הקובץ השני. כלומר ברגע שנסתכל על שם של טרנסקריפט בקובץ השני, נוכל לחפש אותו במילון הראשון ולקשר אותו לשם של גן.

- למילון השני נקרא tpm_per_gene ובו ה-key יהיה השם של הגן וה-value יהיה הערך הכולל של ה-TPM של כל הטרנסקריפטים של הגן.

מעבר על הסקריפט הראשון:

בסקריפט הראשון אנחנו נשתמש בקובץ הראשון של שמות הטרנסקריפטים לשמות הגנים וניצור את המילון הראשון שמקשר בין השמות (gene_name_of_transcript).
טוענים את החבילה של RE כדי שנוכל להשתמש בה (import RE). פותחים את הקובץ הראשון לקריאה עם הפונקציה open שבה רשמים את שם הקובץ ואת האות "r" ושומרים אותו במשתנה file1. מאתחלים את המילון הראשון שנקרא gene_name_of_transcript באמצעות סוגריים מסולסלים. קוראים את השורה הראשונה בקובץ שטענו באמצעות הפונקציה readline(). פונקציה זו מאפשרת לנו לרוץ על השורות בקובץ file1 ולשמור כל פעם שורה אחת בתור המשתנה x. ממשיכים עם לולאת while שממשיכה לרוץ כל עוד True. הלולאה מפסיקה ברגע ש-x כבר לא קיים, כלומר ברגע שסיימנו לקרוא את כל השורות בקובץ file1, כמגיעים לשורה של break. בסוף סוגרים את הקובץ file1 עם הפונקציה close.

בתוך הלולאה: בקובץ המקורי שהוא קובץ טקסט, הרווחים בין 3 העמודות הם רווח עם tab. עכשיו אנחנו רוצים להתחיל למלא את המילון שיצרנו כך שעבור כל שם של טרנסקריפט, נקבל את השם של הגן. לשם כך, אנחנו מנסים לחפש

דפוסים מסוימים שנמצאים בשורה שקראנו מהקובץ, השורה שנקראת x (יש לכם בהערה בקוד דוגמה לשורה בקובץ). אנחנו נרצה למצוא גם את השם של הטרנסקריפט (האיבר הראשון בשורה) וגם את השם של הגן (האיבר האחרון בשורה). כדי למצוא 2 דפוסים באותה שורה באמצעות RE, אנחנו נשתמש ב-Group Extraction. הוא מאפשר לנו למצוא כמה דפוסים במקביל באותה שורה באמצעות סימון שלהם בסוגריים (). הוא שומר את כל הדפוסים שאנחנו מחפשים ואנחנו יכולים לקרוא להם. למשל אנחנו נקרא לדפוס הראשון שחיפשנו עם group(1) ולדפוס השני עם group(2). הדפוס שמחפשים הוא: רשמים ^ שאומר שאנחנו מסתכלים על תחילת השורה. אח"כ יש סוגריים עגולים שמה שבתוכם זה הדפוס שאנחנו מחפשים, כלומר אנחנו מחפשים את הדפוס (+). שזה אומר כל סימן (.). שמופיע יותר מפעם אחת (+). אח"כ יש \t שזה אומר tab. כלומר הדפוס הראשון שאנחנו מחפשים נמצא ממש בתחילת השורה עד ה-tab הראשון (שזה מחזיר לנו את השם של הטרנסקריפט). אח"כ יש עוד פעם נקודה ופלוס שאומר שיכול להיות שם כל דבר, אחרים יש tab ואז יש עוד פעם סוגריים עם נקודה ופלוס שזה עוד דפוס שאנחנו מחפשים שאומר כל סימן יותר מפעם אחת אבל כמובן שמגיע אחרי ה-tab (שזה מחזיר לנו את השם של הגן). בסוף יש \ שאומר שעוברים שורה.

הסוגריים העגולים מסמנים לנו את ה-group וזה מסמן את מה שאנחנו רוצים לחפש ולא מעניין אותנו מה שמחוץ לסוגריים. שומרים את הדפוסים במשתנה match. עכשיו אנחנו יכולים לכתוב match.group() עם מספר בתוך הסוגריים וזה יחזיר לנו את הדפוס במקום שנרצה. למשל match.group(2) יחזיר לנו את השם הגן כי הוא היה הדפוס השני שחיפשנו, הסוגריים השניים.

אם יש match, כלומר אם זה מצא את הדפוסים שחיפשנו אז זה עושה משהו, ואחרת זה מדפיס את x כדי להגיד לנו שיש בעיה. כאשר יש match, אנחנו מציבים במשתנים את הדפוסים שמצאנו, את השם של הטרנסקריפט ואת השם של הגן. בנוסף אנחנו ממלאים את המילון שיצרנו. אנחנו קובעים את ה-key עם סוגריים מרובעים שיהיה השם של הטרנסקריפט ואת ה-value שיהיה השם של הגן.

השורה האחרונה בסקריפט היא רק לבדיקה לראות שהמילון שיצרנו עובד כמו שרצינו.

מעבר על הסקריפט השני:

אחרי שיש לנו את המילון הראשון שמקשר בין כל שם של טרנסקריפט לשם של גן, נשאר לנו לרוץ על הקובץ השני, על כל אחד מהטרנסקריפטים שלו, לבדוק אם הם קיימים במילון הראשון שיצרנו, ואם כן אז להתאים אותם לשמות של הגן לפי המילון, לסכום את כל ערכי ה-TPM של כל הטרנסקריפטים של אותו גן ולשומר את המידע הזה במילון חדש.

הסקריפט השני מקבל את הקובץ השני ואת המילון הראשון שיצרנו ומחזיר מילון שנקרא tpm_per_gene שאומר את סכום ה-TPM של כל גן. ה-key הוא השם של הגן וה-value הוא מספר.

גם כאן טוענים את החבילה של RE, טוענים את הקובץ השני ויוצרים את המילון השני עם סוגריים מסולסלים. באמצעות לולאת for שרצה על כל אחד משמות של הגנים במילון הראשון, מאתחלים את המילון השני כך שכל שם של גן מקבל את הערך 0. קוראים בהתחלה לשורה הראשונה בקובץ עם readline ומיד אח"כ לשנייה מכיוון שהשורה הראשונה היא כותרות. אנחנו רוצים לספור כמה טרנסקריפטים חופפים היו בין הקבצים ולכן יוצרים 2 משתנים שמאתחלים ל-0. גם פה נכנסים ללולאת while שרצה כל עוד יש שורות לקרוא בקובץ השני. משתמשים ב-RE כמו מקודם, הפעם כדי למצוא את שם הטרנסקריפט (עמודה ראשונה) ואת ערך ה-TPM שלו (עמודה אחרונה). כמובן שיש דרכים נוספות לכתוב את זה אבל זאת דרך אחת קלה ופשוטה. אם מצאנו את הדפוסים שחיפשנו, אנחנו שומרים אותם במשתנים transcript_name ו-tpm. יכולים להיות טרנסקריפטים שמופיעים בקובץ אחד ולא מופיעים בקובץ השני ולכן משתמשים בפונקציה get שתבדוק אם הטרנסקריפט שקראנו בקובץ השני נמצא במילון שיצרנו (כלומר מופיע גם בקובץ הראשון). אם הוא מופיע, אנחנו מחלצים מהמילון את השם של הגן שמתאים לטרנסקריפט שיש לנו ומכניסים את ערך ה-TPM שלו למילון החדש. שימו לב שהפעם ה-value יהיה מספר והוא יהיה הסכום של כל ערכי ה-TPM של הטרנסקריפטים של הגן שכבר הכנסנו למילון פלוס ערך ה-TPM של הטרנסקריפט הנוכחי של הגן. אם הטרנסקריפט הנוכחי נמצא במילון, מוסיפים 1 ל-

transcript_found, ואם הוא לא נמצא, מוסיפים 1 ל-transcript_not_found. ברגע שמסיימים לרוץ על כל השורות בקובץ, סוגריים אותו, ואפשר להדפיס את המשתנים שיצרנו ודוגמה לערך במילון.

מעבר על הסקריפט השלישי:

עכשיו יש לנו מילון שמכיל את שמות הגנים ואת ערך ה-TPM של כל גן. יוצרים קובץ טקסט שנקרא ReadPerGene, הפעם עם "w" ולא עם "r" כדי שנוכל לכתוב בו, שמכיל את כל הגנים עם ה-TPM שלהם בסדר יורד, כלומר למעלה יהיה הגן עם הכי הרבה עותקים בדגימה. עושים זאת באמצעות הפונקציה sorted ומסדרים לפי ערכי ה-TPM. כל גן במילון שממוין מהגבוה לנמוך, שומרים בקובץ txt ובסוף סוגרים את הקובץ.

באופן הזה קיבלנו את וקטור רמות הביטוי של הגנים בדגימה בסדר יורד (Output file).

מקבלים שיש הכי הרבה מהגן Lars2. במקרים כאלה, כדאי לחשוד שיכול להיות שאולי יש איפשהו בעיה. אם נסתכל על הגן הזה ב-UCSC (כמו שראינו בתרגול 2), נעשה זום אין לסוף שלו (3' UTR) ונסתכל באזור הזה על הערוץ שמראה אזורים חוזרניים (אזורים שחוזרים על עצמם המון פעמים בגנום) שנקרא RepeatMasker, אנחנו נראה שיש בו אזור חוזרני ואם נסתכל יותר לעומק על האזור החוזרני הזה (נלחץ עליו), נראה שהוא rRNA (ribosomal RNA) שזה אזור שקשור לריבוזום ולא באמת קשור לתפקיד של הגן ולכן הוא משועתק המון פעמים בלי קשר לצורך בגן הזה. אנחנו מבינים שהכמות הרבה שיש מהגן הזה לא באמת אומרת שהגן הזה כ"כ חשוב. אפשר גם לראות בערוץ של RefSeq שהורידו שם את האזור הזה מהגן.